02 — Strings and Lists

These appear in **every single question**. Fluency here = speed on exam day.

> **Big idea to hold onto:** a **string is just a list of characters**, and both strings and lists are **sequences** — so almost everything below works the same on both. `"cat"[0]` is `"c"`, just like `[10,20,30][0]` is `10`. Learn a trick once, use it on both.

---

1. Indexing and slicing (works on strings AND lists)

Indexing = grabbing ONE item by its position. **Positions start at 0**, not 1.

```
 a  m  a  l  i  t  e  c  h
 0  1  2  3  4  5  6  7  8      ← positions from the front
-9 -8 -7 -6 -5 -4 -3 -2 -1      ← positions from the back
```

```
s = "amalitech"
s[0]      # 'a'  — first item (position 0)
s[-1]     # 'h'  — last item (negative counts from the end)
len(s)    # 9    — how many items
```

Slicing = grabbing a RANGE of items: `s[start:stop]`. The rule that trips up everyone: **it includes `start` but STOPS BEFORE `stop`** (same "stops one early" idea as `range`).

```
s[2:5]    # 'ali'  — positions 2,3,4  (NOT 5!)
s[:4]     # 'amal' — from the start up to 4
s[5:]     # 'tech' — from 5 to the end
```

A slice can take a **third number, the step** — how many to jump each time:

```
s[::2]    # 'aaiec'    — every 2nd character (step 2)
s[::-1]   # 'hcetilama' ← REVERSE (step -1 walks backwards). MEMORI
```

Read `s[::-1]` as: "whole string, walked backwards." It's the standard way to reverse anything — you already used it in `reverse_number`.

---

2. String methods you must know

A **method** is a built-in action you call on a string with a dot: `s.upper()`.

```
s.upper()   s.lower()          # change case: "Hi".upper() → "HI"
s.strip()                      # remove spaces at both ends: "  hi
s.split()                      # sentence → list of words: "a b  c"
s.split(",")                   # split on a comma: "a,b,c".split(",
"-".join(['a','b'])            # list of strings → one string: 'a-b
s.count("a")                   # how many times 'a' appears
s.replace("a", "o")            # NEW string with every 'a' turned i
s.find("li")                   # position of first "li", or -1 if n
s.startswith("ama")            # True/False — does it begin with th
s.endswith("ech")             # True/False — does it end with this?
c.isdigit()  c.isalpha()       # is this character a digit? a letter
c.islower()  c.isupper()       # is it lowercase? uppercase?
ord('a')   # 97 — letter → its number code
chr(98)    # 'b' — number code → letter
```

> **Trap — a string method NEVER changes the original.** It returns a NEW string. So `s.upper()` alone does nothing lasting — you must **keep the result**: `s = s.upper()`. (See section 2b on why.)

2b. Strings are immutable — what that means

Immutable = frozen once created; you can never edit it in place.

```
s = "cat"
s[0] = "b"        # ❌ CRASH (TypeError) — you cannot change a lett
```

Think of a string as **ink on paper**: to change one letter you rewrite the whole word. So the way to "modify" a string is to **build a new one**. Two common ways:

```
# Way 1: piece it together with slicing
s = "X" + s[1:]          # replace first char: keep "X", then everyt

# Way 2 (the reliable recipe): string → list → edit → join back
lst = list(s)            # turn into a list of chars (lists ARE edi
lst[0] = "X"
s = "".join(lst)         # fuse the list back into a string
```

That **string → list → edit → join** recipe is the standard answer whenever a problem says "replace/remove/change characters."

Handy trick: **alphabet position.** `ord(c) - ord('a')` gives a=0, b=1, ... (add 1 if you want a=1, b=2, ...). This is the base of Caesar-cipher problems.

---

3. List operations

A **list** is an ordered, EDITABLE collection: `[3, 1, 4]`.

```
nums = [3, 1, 4]
nums.append(5)        # add to the end        → [3,1,4,5]
nums.pop()            # remove & return LAST  → returns 5,  nums = [
nums.pop(0)           # remove & return FIRST → returns 3,  nums = [
nums.insert(1, 99)    # insert 99 at position 1
nums.index(4)         # position of the first 4
nums.remove(3)        # delete the first 3 BY VALUE (not position)
nums.count(1)         # how many 1s
4 in nums             # True/False — is 4 present?
nums2 = nums[:]       # a real COPY (see the trap below)
[0] * 5               # [0,0,0,0,0] — quick way to make a filled lis
list(range(5))        # [0,1,2,3,4]
```

> **Trap 1 — `.sort()` and `.reverse()` return `None`.** They edit the list *in place* and hand back nothing. So `a = nums.sort()` sets `a` to `None` — a classic exam-killer.
>
> - Want to sort in place: `nums.sort()` (then use `nums`).
> - Want a new sorted list, original untouched: `new = sorted(nums)`.

> **Trap 2 — `b = a` does NOT copy a list.** It makes `b` a second name for the *same* list, so editing `b` also changes `a`. To get an independent copy use `b = a[:]`.

---

4. List comprehensions — your speed superpower

A **comprehension** builds a new list in one line. It's a `for` loop folded up.

```
squares = [x * x for x in range(5)]          # [0,1,4,9,16]
```

Read it left-to-right as a sentence: **"give me `x*x`, for each `x` in `range(5)`."** The long form is identical:

```
squares = []
for x in range(5):
    squares.append(x * x)
```

Add an `if` at the end to **filter** (keep only some items):

```
evens  = [x for x in nums if x % 2 == 0]     # keep only even numbe
words  = [w.upper() for w in sentence.split()]   # uppercase every
digits = [int(d) for d in str(n)]            # the digits of a numb
```

General shape: **`[ what_to_keep for item in collection if condition ]`** (the `if` is optional).

---

5. zip — walk two lists together

`zip` pairs up items from two lists, position by position, so one loop sees both:

```
names = ["a", "b"]; scores = [90, 80]
for name, score in zip(names, scores):
    ...        # first loop: name="a", score=90 ; then name="b", sc
```

A neat use — **compare each item with its neighbor** by zipping a list with itself shifted by one:

```
for prev, cur in zip(nums, nums[1:]):
    ...        # prev,cur = each adjacent pair: (nums[0],nums[1]),
```

---

6. Classic exam patterns (with plain-English walkthroughs)

Palindrome check — reads the same forwards and backwards (`"level"`, `"radar"`):

```
def is_palindrome(s):
    return s == s[::-1]        # "is s equal to s reversed?" → True,
```

For `"radar"`: `s[::-1]` is `"radar"`, so `"radar" == "radar"` → `True`. (If the problem says ignore case/spaces, clean first: `s = s.lower().replace(" ", "")`.)

Adjacent element product (a real Q2) — biggest product of two side-by-side numbers:

```
def adjacent_max_product(nums):
    return max(nums[i] * nums[i+1] for i in range(len(nums) - 1))
```

Why `len(nums) - 1`? Because `nums[i+1]` would fall off the end on the last index. When you pair neighbors, always loop one short.

Alternating sums (real GCA) — add the even-position items and odd-position items separately:

```
def alternating_sums(a):
    return [sum(a[::2]), sum(a[1::2])]
```

`a[::2]` = positions 0,2,4,... (starts at 0 = "even"); `a[1::2]` = positions 1,3,5,... For `[10,20,30,40,50]` → `[10+30+50, 20+40]` = `[90, 60]`.

Longest word in a sentence:

```
def longest_word(sentence):
    return max(sentence.split(), key=len)
```

`key=len` tells `max`: *don't compare the words themselves — compare their lengths.* So it returns the longest word. **`key=` works in `max`, `min`, and `sorted`** and is exam gold:

```
min(words, key=len)              # shortest word
sorted(words, key=len)           # words ordered short → long
sorted(nums, key=abs)            # sort by distance from zero
```

Caesar shift by k — move each letter forward k places in the alphabet, wrapping z→a:

```
def shift(s, k):
    out = []
    for c in s:
        out.append(chr((ord(c) - ord('a') + k) % 26 + ord('a')))
    return ''.join(out)
```

Read the inner formula in 4 steps, for one letter `c`:

1. `ord(c) - ord('a')` → letter's position 0–25 (a=0, b=1, ...)
2. `+ k` → shift it forward
3. `% 26` → **wrap around** the 26-letter cycle (position 27 → 1). Same `i % k` wrap trick from lesson 01.
4. `+ ord('a')` then `chr(...)` → turn the position back into a real letter.

Build a string in a loop — because strings are immutable, don't grow one with `s += c` in a big loop. Instead **collect pieces in a list, then `''.join(...)` once at the end** (the Caesar code above does exactly this). Cleaner and faster.

Exercises (in `practice/q2_data_manipulation.py`)

1. `all_longest_strings(words)` — return all strings of maximum length, in order.
2. `add_border(picture)` — surround a list of strings with * (real GCA problem).
3. `sorted_squares(nums)` — square every number, return sorted.
4. `is_anagram(a, b)` — do two words use exactly the same letters?
5. `first_digit(s)` — return the first character in a string that is a digit.

Which tool for each? 1 → `max(..., key=len)` then a filter comprehension. 2 → build new strings with `"*" + row + "*"` and add top/bottom border rows. 3 → `sorted([x*x for x in nums])`. 4 → compare `sorted(a) == sorted(b)`. 5 → loop and return the first `c` where `c.isdigit()`.